

IA: MACHINE LEARNING FOUNDATIONS

Multilayer Perceptron

Adrián Sánchez Morales

AGENDA

1. Introduction
2. Single Layer Perceptron
3. Multilayer Perceptron
   - 3.1. Universal Approximation
   - 3.2. Backpropagation
   - 3.3. Activation Functions
   - 3.4. Other Considerations
   - 3.5. Training in Practice
4. Tensorflow & Keras
5. Summary



1. INTRODUCTION

- Deep Learning, a group of Machine Learning methods that has its roots dating back to the 1940's.

- It gained attention in the last decades for its ground-breaking application in areas like image classification, speech recognition, and machine translation.

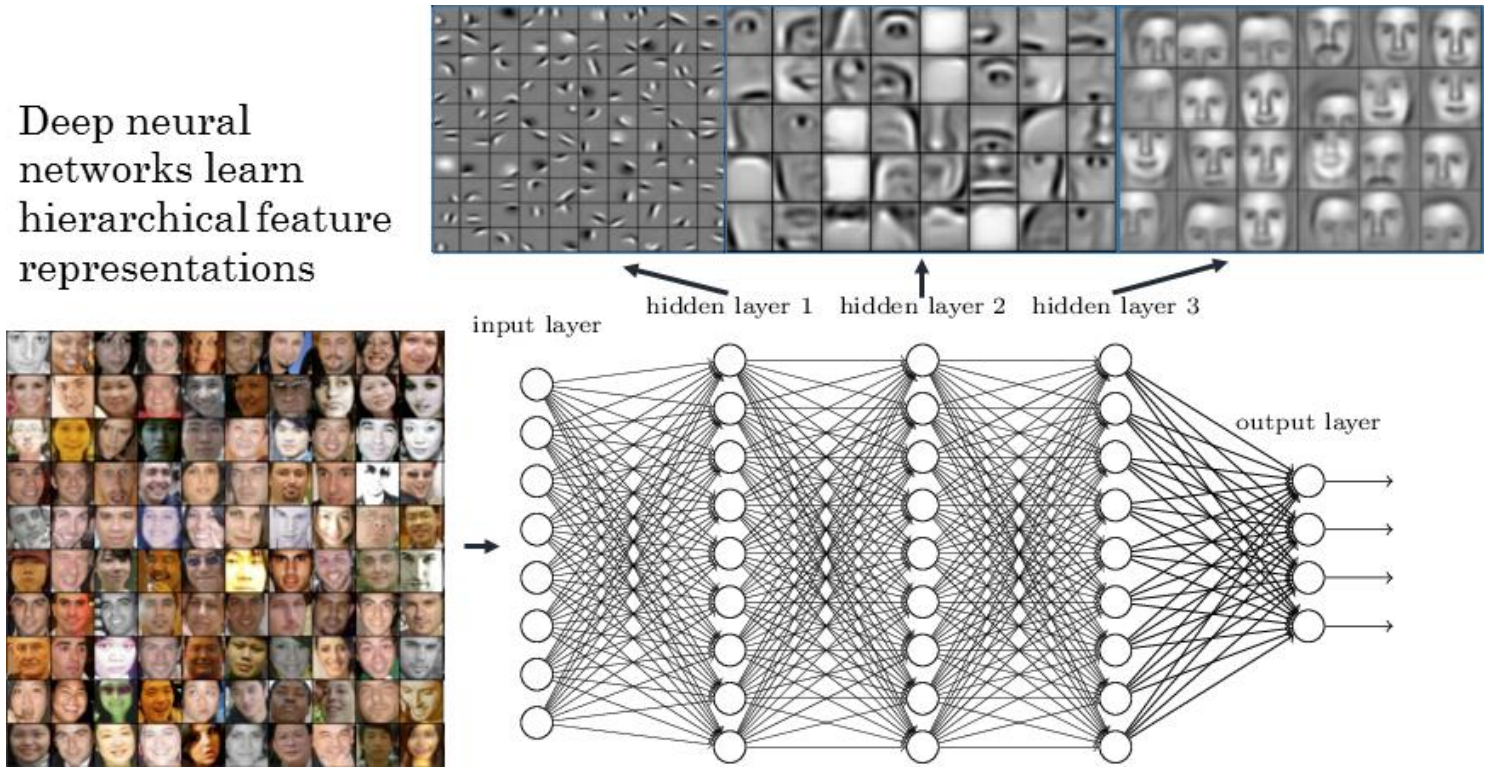
- Deep Learning is Game Changing:
  - In traditional ML it is frequently required a domain knowledge
  - The quality of a Machine Learning model depends on the quality of the dataset, but also on how well features encode the patterns in the data

- Deep Learning algorithms use Artificial Neural Networks as their main structure. What sets them apart from other algorithms is that they don't require expert input during the feature design and engineering phase. Neural Networks can learn the characteristics of the data.

1. INTRODUCTION

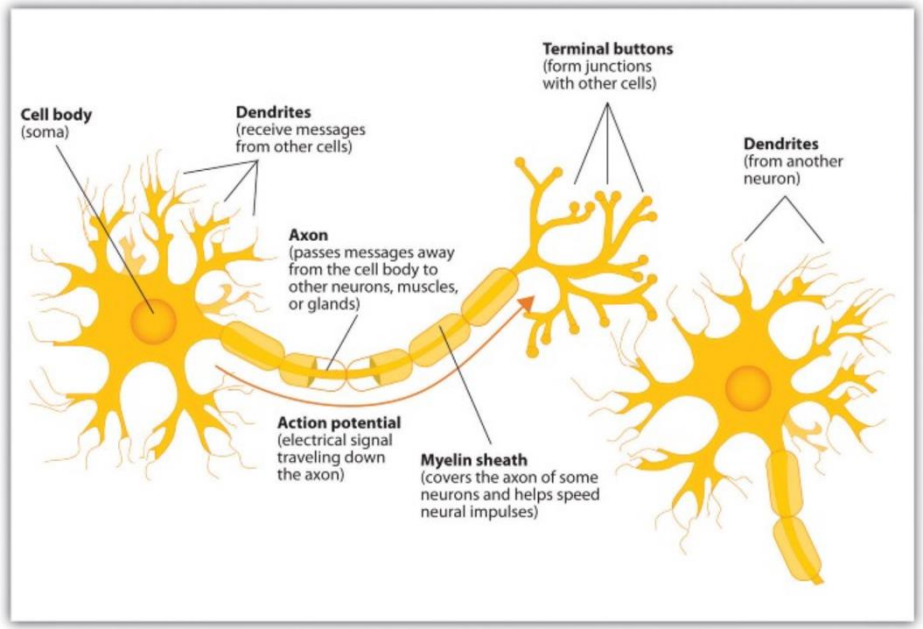
- Deep Learning algorithms take in the dataset and learn its patterns, they learn how to represent the data with features they extract on their own.

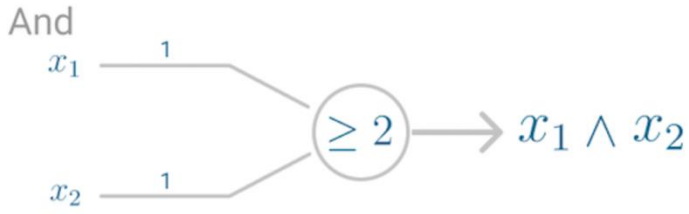
- Then they combine different representations of the dataset, each one identifying a specific pattern or characteristic, into a more abstract, high-level representation of the dataset.

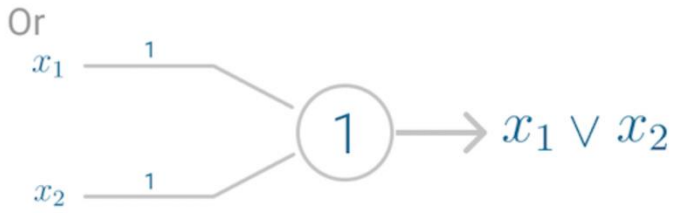


1. INTRODUCTION

- It's all started with a **neuron**.
- In the early 1940's Warren McCulloch, a neurophysiologist, teamed up with logician Walter Pitts to create a model of how brains work. It was a simple linear model.
- The first application replicated logic gates where the weights were set beforehand.
- It was only a decade later that Frank Rosenblatt extended this model and created an algorithm that could learn the weights in order to generate an output.
- It was the **perceptron**.

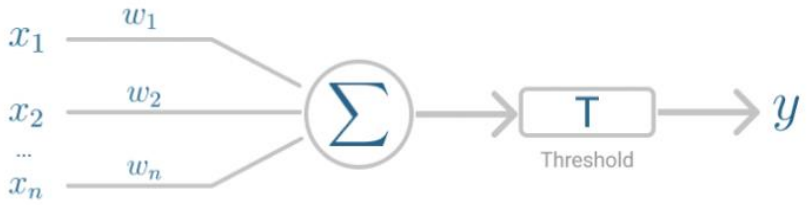






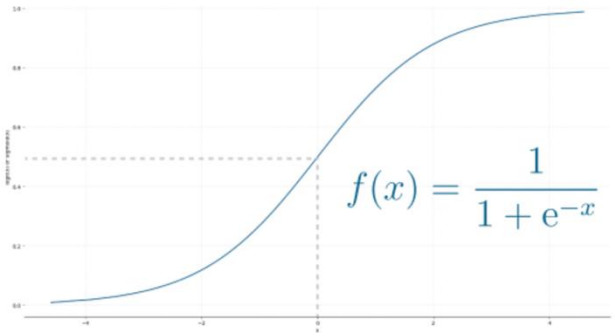
1. INTRODUCTION

- Perceptron is a machine with a basic unit of computation, a **neuron**.

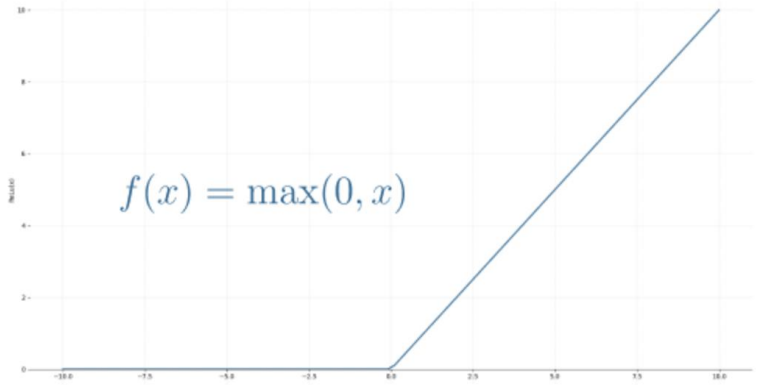


$$y = \begin{cases} 1, \text{if } \overbrace{\sum_i w_i x_i}^{\text{weighted sum}} - \underbrace{T}_{\text{threshold}} > 0 \\ 0, \text{otherwise} \end{cases}$$

- It can be used for binary classification by using Sigmoid as an activation function.



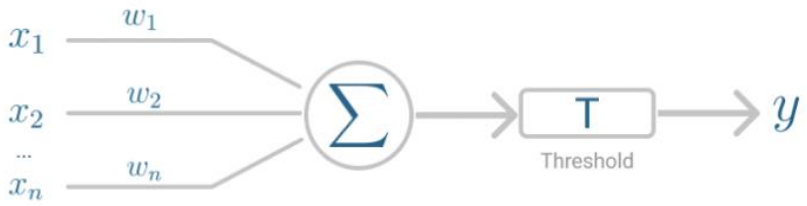
- If you look at Deep Learning papers, you'll see one of the most popular is Rectified Linear Unit (ReLU).



WHY?

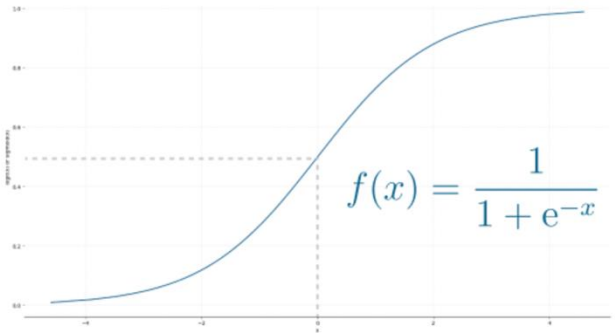
1. INTRODUCTION

- Perceptron is a machine with a basic unit of computation, a **neuron**.

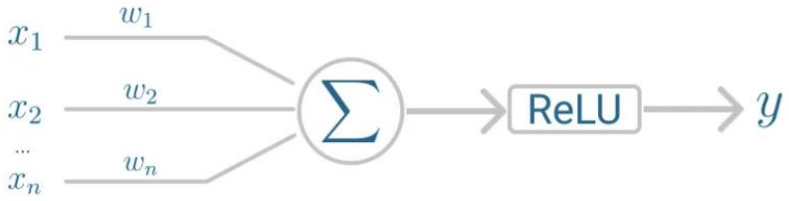


$$y = \begin{cases} 1, \text{if } \overbrace{\sum_i w_i x_i}^{\text{weighted sum}} - \underbrace{T}_{\text{threshold}} > 0 \\ 0, \text{otherwise} \end{cases}$$

- It can be used for binary classification by using Sigmoid as an activation function.



- If you look at Deep Learning papers, you'll see one of the most popular is Rectified Linear Unit (ReLU).
  - It allows better optimization using Gradient Descent
  - More efficient computation



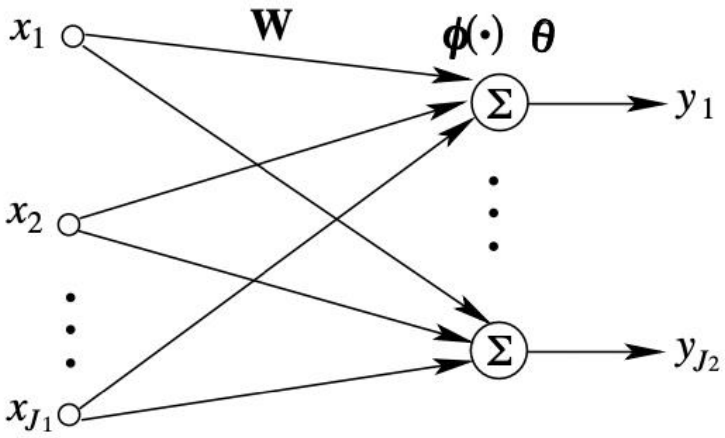
AGENDA

1. Introduction
2. Single Layer Perceptron
3. Multilayer Perceptron
   - 3.1. Universal Approximation
   - 3.2. Backpropagation
   - 3.3. Activation Functions
   - 3.4. Other Considerations
   - 3.5. Training in Practice
4. Tensorflow & Keras
5. Summary



2. SINGLE LAYER PERCEPTRON

- Single layer perceptron (SLP) is a set of neurons that can be used to classify problems with more than two classes



$$net = \mathbf{W}^T \boldsymbol{x} - \boldsymbol{\theta}$$

$$\hat{\boldsymbol{y}} = \phi(\boldsymbol{net})$$

$\mathbf{W}$

$\boldsymbol{x}$

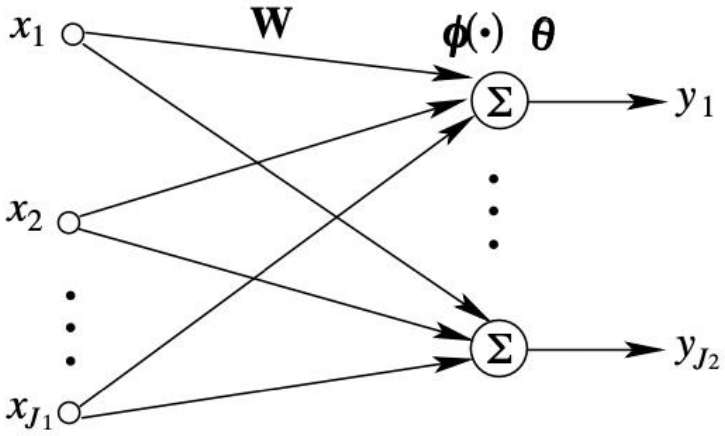
$\boldsymbol{\theta}$

ϕ

- Can a neuron be considered a SLP?

2. SINGLE LAYER PERCEPTRON

- Single layer perceptron (SLP) is a set of neurons that can be used to classify problems with more than two classes



$$net = \mathbf{W}^T \boldsymbol{x} - \boldsymbol{\theta}$$

$$\hat{\boldsymbol{y}} = \boldsymbol{\phi}(\boldsymbol{net})$$

$\mathbf{W}$	Weights
$\boldsymbol{x}$	Input
$\boldsymbol{\theta}$	Bias
$\boldsymbol{\phi}$	Activation Function

- Can a neuron be considered a SLP? ⟶ Binary problems
- There exist some methods to train the perceptron ⟶ Gradient Descent

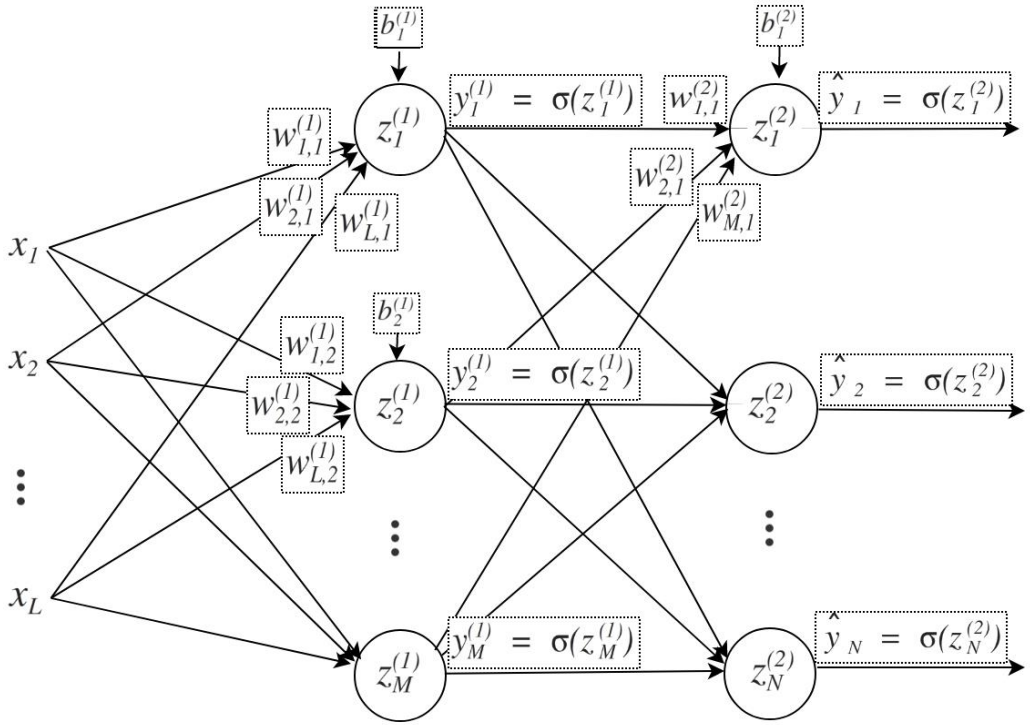
AGENDA

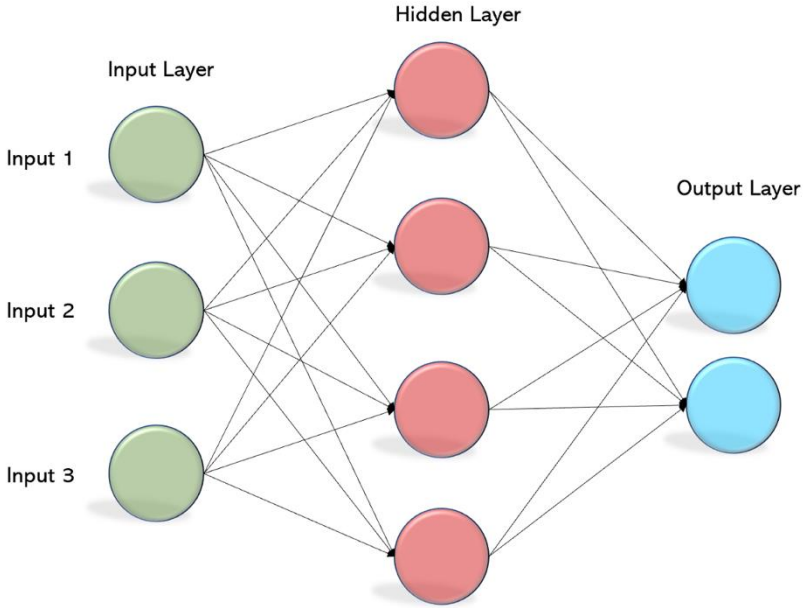
1. Introduction
2. Single Layer Perceptron
3. Multilayer Perceptron
   - 3.1. Universal Approximation
   - 3.2. Backpropagation
   - 3.3. Activation Functions
   - 3.4. Other Considerations
   - 3.5. Training in Practice
4. Tensorflow & Keras
5. Summary



3. MULTILAYER PERCEPTRON

- MLPs are feedforward networks with one or more layers of units between the input and output layers. It is the original form of artificial neural networks.





3. MULTILAYER PERCEPTRON

- Researchers often use Multilayer Perceptrons to solve complex problems stochastically.

- This classifier is highly effective in arranging and categorizing input data into different classes, allowing probability-based predictions and classifying items into multiple categories.

- Multilayer Perceptrons have the advantage of learning non-linear models and the ability to train models in real-time (online learning).

- Other advantages:
  - It can be used to solve complex nonlinear problems.
  - It handles large amounts of input data well.
  - Makes quick predictions after training.
  - A single-layer perceptron can only learn linear functions, but Multilayered Perceptrons can also learn non-linear functions.

3. MULTILAYER PERCEPTRON

- MLPs have been used in multiple fields such as speech recognition, image recognition, and machine translation software, but thereafter faced strong competition from much simpler support vector machines.
- Interest in backpropagation networks returned due to the success of deep learning.
- They became a key piece of business applications. Between others:
  - Social Media – data compression in Youtube, Spotify, Instagram, etc.
  - Cybersecurity – data encryption for personal information or data transmission between platforms
  - Autonomous driving
  - Healthcare, industry, retail, insurance, etc.

3. MULTILAYER PERCEPTRON

3.1. UNIVERSAL APPROXIMATION

- The hidden layers and output layer use a non-linear activation function that models the behavior of the neurons by combining input with the weights of the neurons and adding bias.

- Thanks to these nonlinearities used in the nodes ⟶ **Universal Approximator**

QUESTION

Could anyone explain the universal approximation theorem in MLPs?

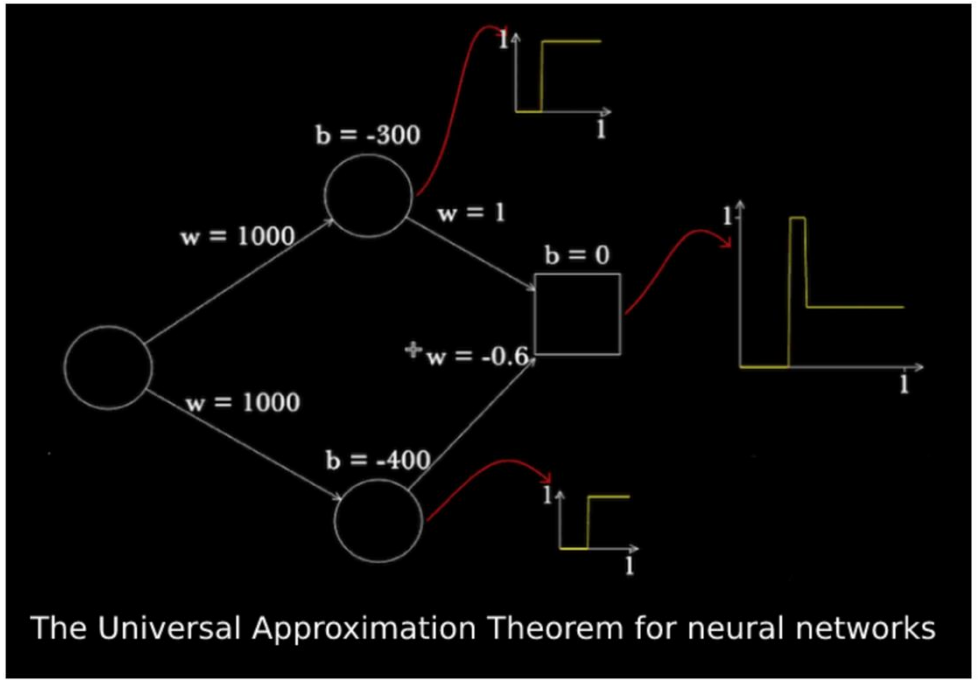
3. MULTILAYER PERCEPTRON

3.1. UNIVERSAL APPROXIMATION

- The hidden layers and output layer use a non-linear activation function that models the behavior of the neurons by combining input with the weights of the neurons and adding bias.

- Thanks to these nonlinearities used in the nodes ⟶ **Universal Approximator**

Given any continuous function at all, no matter how complicated it might be it's always possible to find an artificial neural network which can approximate that function.

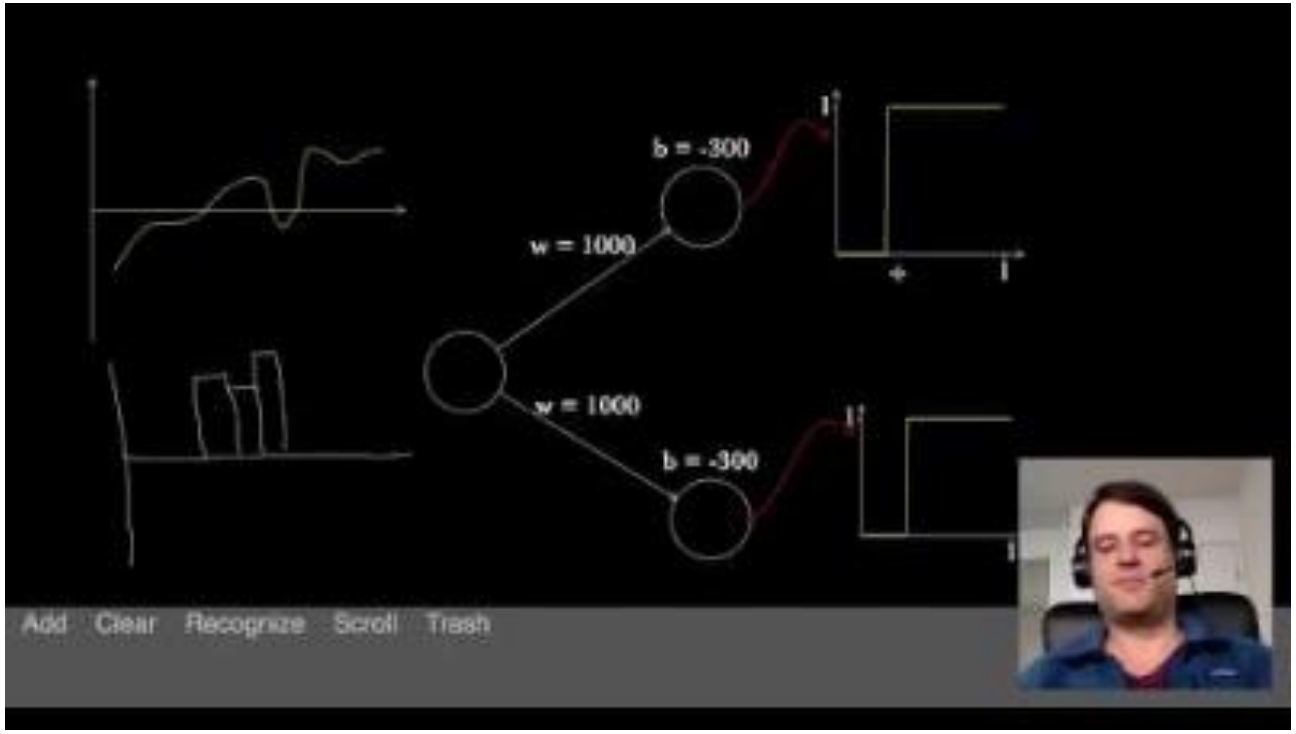


3. MULTILAYER PERCEPTRON

3.1. UNIVERSAL APPROXIMATION

- The hidden layers and output layer use a non-linear activation function that models the behavior of the neurons by combining input with the weights of the neurons and adding bias.

- Thanks to these nonlinearities used in the nodes ⟶ **Universal Approximator**



3. MULTILAYER PERCEPTRON

3.1. UNIVERSAL APPROXIMATION

- The hidden layers and output layer use a non-linear activation function that models the behavior of the neurons by combining input with the weights of the neurons and adding bias.
- Thanks to these nonlinearities used in the nodes ⟶ **Universal Approximator**

Play with a Neural Network

3. MULTILAYER PERCEPTRON

3.2. BACKPROPAGATION – INTRO

- BP learning is the most popular learning rule for performing supervised learning tasks.
- It **uses a gradient-search technique** to minimize an error function between the desired and actual network outputs. Thanks to the BP algorithm, MLP can be extended to many layers.
- The BP algorithm **propagates backward the error** between the desired signal and the network output through the network.
- After providing an input pattern, the output of the network is then compared with a given target pattern and the error of each output unit calculated.
- This error signal is propagated backward, and a closed-loop control system is thus established.
- The weights can be adjusted by a gradient-descent based algorithm.

3. MULTILAYER PERCEPTRON

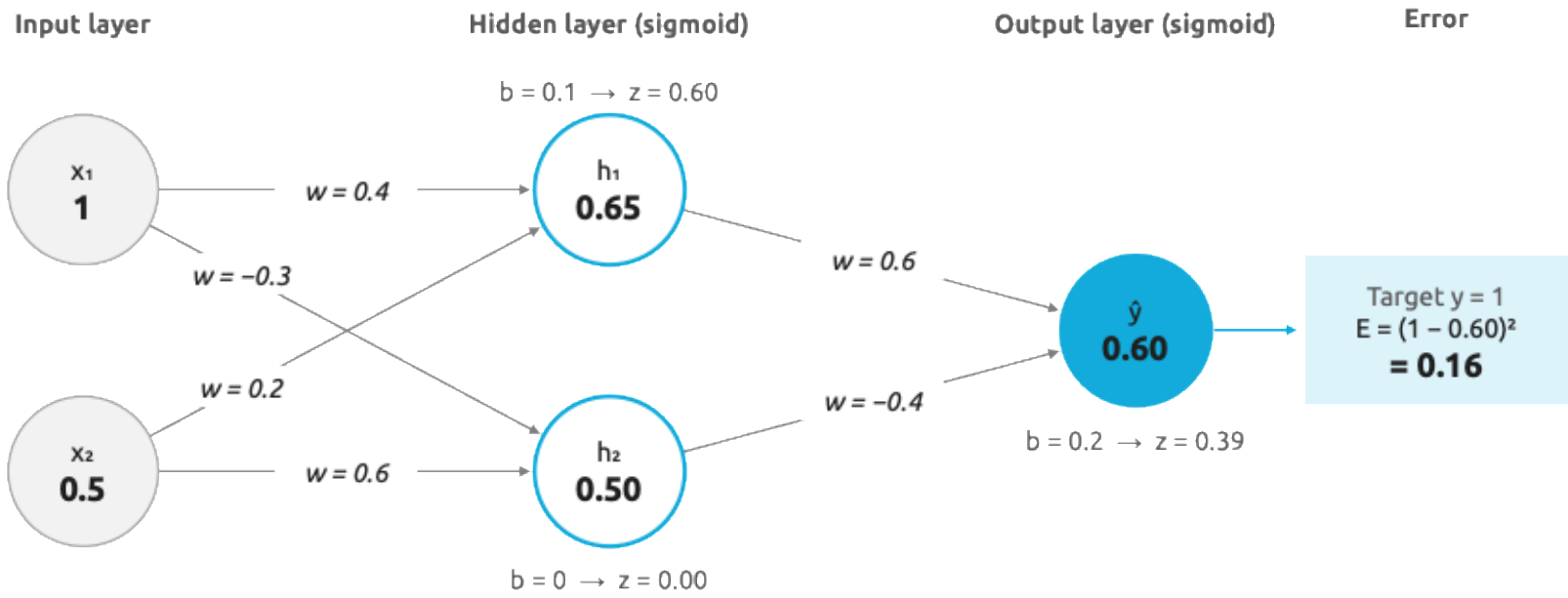
3.2. BACKPROPAGATION

- **Forward prop**: before computing gradients, the network makes a prediction layer by layer:

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)} \qquad a^{(l)} = \sigma(z^{(l)}) \qquad with\ a^{(0)} = x\ \ and\ \ a^{(L)} = \hat{y}$$

- The error E(y, ŷ) is then propagated **backwards** (next slides) to update every W and b.

Worked example: x = (1, 0.5), target y = 1, sigmoid activations



Squared error: $E = (y - \hat{y})^2 = (1 - 0.60)^2 = \mathbf{0.16}$. This is the value backpropagation will reduce.

3. MULTILAYER PERCEPTRON

3.2. BACKPROPAGATION

- A nonlinear activation function, differentiable almost everywhere, is needed: logistic and tanh are the classic choices; ReLU (non-differentiable only at 0) is today's default.
- MSE/SE are commonly used as error functions to be optimized:

$$E(y, \hat{y}) = \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

- Using gradient descent:

$$\Delta \mathbf{W} = -\eta \frac{\partial E}{\partial \mathbf{W}}$$

- Partial derivative of the error with respect to the output:

$$\frac{\partial E}{\partial \hat{y}_k} = \frac{\partial}{\partial \hat{y}_k} \left(\sum_{i=1}^{N} (y_i - \hat{y}_i)^2 \right) = \frac{\partial}{\partial \hat{y}_k} (y_k - \hat{y}_k)^2 = -2(y_k - \hat{y}_k)$$

3. MULTILAYER PERCEPTRON

3.2. BACKPROPAGATION

- Applying the chain rule:

$$\frac{\partial E}{\partial z_i^{(2)}} = \frac{\partial E}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial z_i^{(2)}} = -2(y_i - \hat{y}_i)\sigma'(z_i^{(2)})$$

σ' is the derivative of the neuron activation function

- That's the local gradient with respect to the *i*-th neuron of output layer and, to not make the formulas too long, it will be simply indicated as:

$$\frac{\partial E}{\partial z_i^{(2)}} = \delta_i^{(2)}$$

- And using the chain rule again, we calculate the partial derivative of the error with respect to the weight W of output layer:

$$\frac{\partial E}{\partial w_{j,i}^{(2)}} = \frac{\partial E}{\partial z_i^{(2)}} \frac{\partial z_i^{(2)}}{\partial w_{j,i}^{(2)}} = \delta_i^{(2)} \frac{\partial}{\partial w_{j,i}^{(2)}} \left(\sum_{k=1}^{M} w_{k,i}^{(2)} y_k^{(1)} + b_i^{(2)} \right) = \delta_i^{(2)} y_j^{(1)}$$

3. MULTILAYER PERCEPTRON

3.2. BACKPROPAGATION

- Once we have these results, we can update output weights and similarly the output bias:

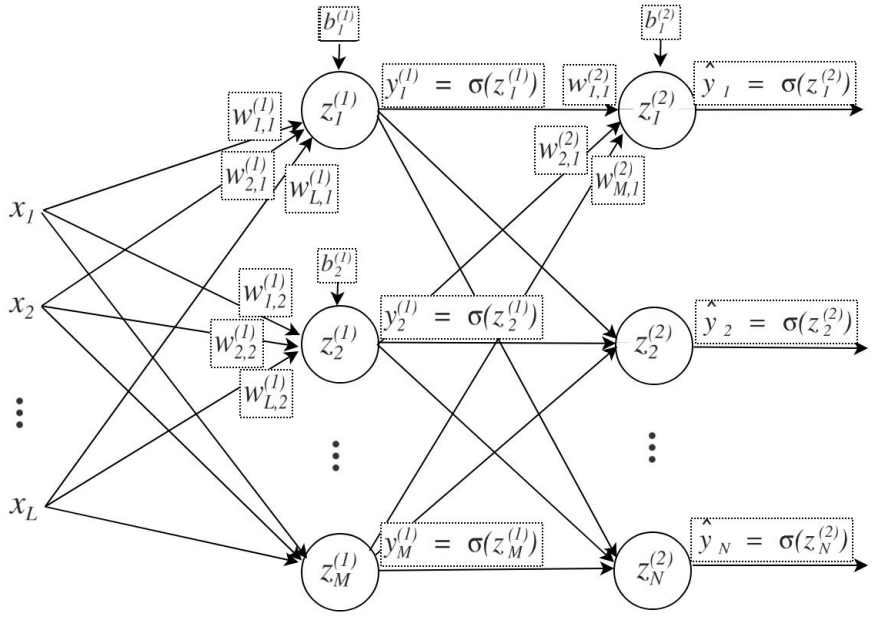
$$w_{j,i}^{(2)} \leftarrow w_{j,i}^{(2)} - \eta \delta_i^{(2)} y_j^{(1)} \qquad b_i^{(2)} \leftarrow b_i^{(2)} - \eta \delta_i^{(2)}$$

- Doing the same for the hidden layer and realizing that y_i interferes in all neurons in the next layer:

$$\frac{\partial E}{\partial y_i^{(1)}} = \sum_{j=1}^{N} \frac{\partial E}{\partial z_j^{(2)}} \frac{\partial z_j^{(2)}}{\partial y_i^{(1)}}$$

$$\frac{\partial z_j^{(2)}}{\partial y_i^{(1)}} = \frac{\partial}{\partial y_i^{(1)}} \left(\sum_{k=1}^{M} w_{k,j}^{(2)} y_k^{(1)} + b_j^{(1)} \right) = w_{i,j}^{(2)}$$

$$\frac{\partial E}{\partial y_i^{(1)}} = \sum_{j=1}^{N} \delta_j^{(2)} w_{i,j}^{(2)}$$



3. MULTILAYER PERCEPTRON

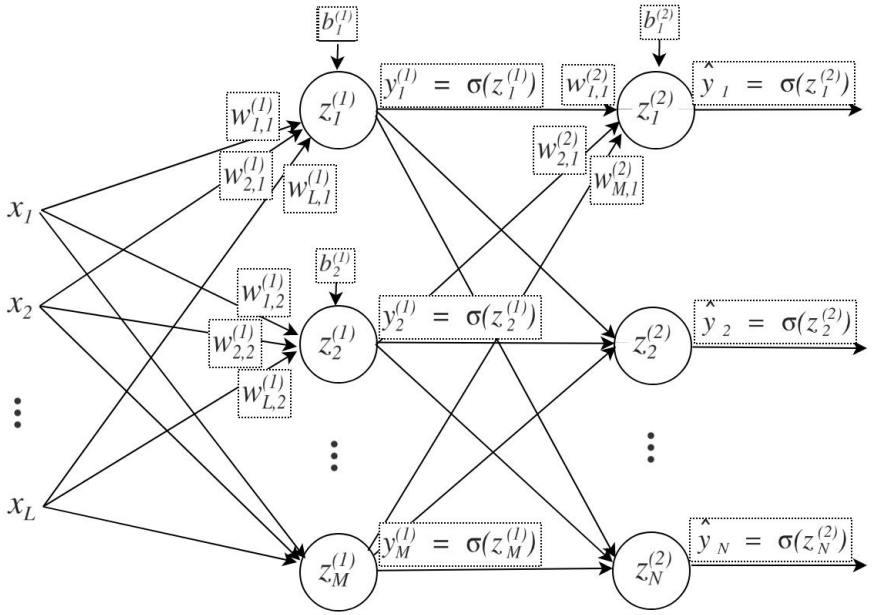
3.2. BACKPROPAGATION

- Going ahead:

$$\frac{\partial E}{\partial z_i^{(1)}} = \frac{\partial E}{\partial y_i^{(1)}} \frac{\partial y_i^{(1)}}{\partial z_i^{(1)}} = \left(\sum_{j=1}^{N} \delta_j^{(2)} w_{i,j}^{(2)} \right) \sigma'(z_i^{(1)}) = \delta_i^{(1)}$$

$$\frac{\partial E}{\partial w_{j,i}^{(1)}} = \frac{\partial E}{\partial z_i^{(1)}} \frac{\partial z_i^{(1)}}{\partial w_{j,i}^{(1)}} = \delta_i^{(1)} \frac{\partial}{\partial w_{j,i}^{(1)}} \left(\sum_{k=1}^{L} w_{k,i}^{(1)} x_k + b_i^{(1)} \right) = \delta_i^{(1)} x_j$$

$$w_{j,i}^{(1)} \leftarrow w_{j,i}^{(1)} - \eta \delta_i^{(1)} x_j$$

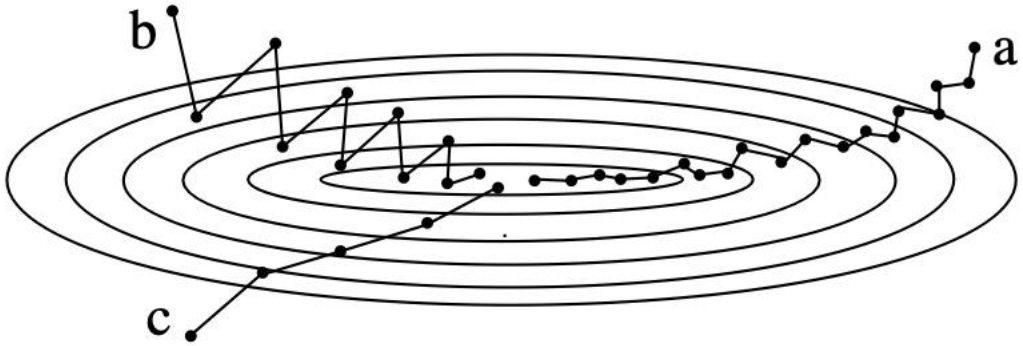


- If there were more layers, the process would continue.

3. MULTILAYER PERCEPTRON

3.2. BACKPROPAGATION

- Learning rate will influence the convergence speed of the backpropagation process:



- Sometimes, the BP algorithm can be improved by adding a momentum term. This algorithm is called ***BP with momentum***:

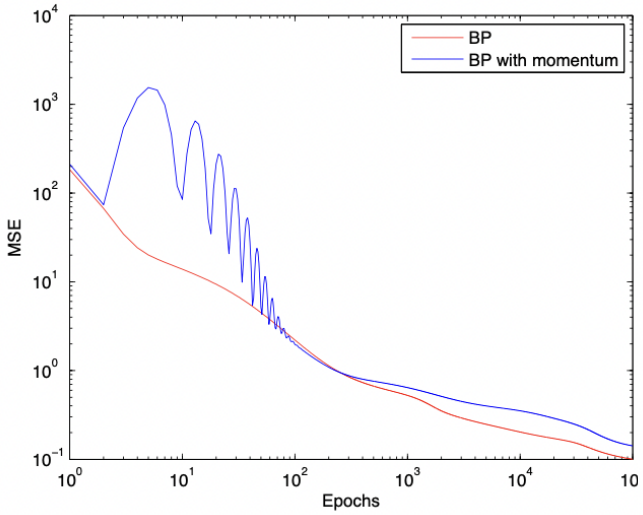
$$\Delta_p \mathbf{W}(t) = -\eta \frac{\partial E_p(t)}{\partial \mathbf{W}(t)} + \alpha \Delta \mathbf{W}(t-1)$$

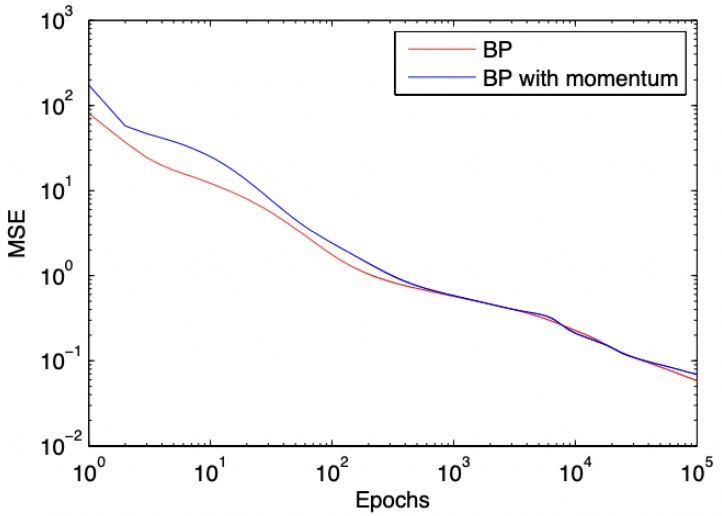
3. MULTILAYER PERCEPTRON

3.2. BACKPROPAGATION

- Incremental learning (**stochastic gradient descent**) and batch learning are two methods for BP learning.
- In incremental learning, the training samples are presented to the network sequentially.
- In batch learning, the optimization objective is E, and the weight update is performed at the end of an epoch.
- For sufficiently small learning rates, incremental learning approaches batch learning and the two methods produce the same results.

Batch Learning





Incremental Learning

3. MULTILAYER PERCEPTRON

3.2. BACKPROPAGATION – MINI-BATCHES AND OPTIMIZERS

- **Mini-batch learning** sits between both: weights are updated after every batch of B samples (typically 32–256). It is the default in practice.
- Modern optimizers adapt the step size of each weight automatically:

Optimizer	How it updates the weights	When to use it
SGD + momentum	Accumulates past gradients to smooth and speed up the descent	Strong baseline when well tuned
RMSprop	Divides each step by a running average of squared gradients (per-weight learning rate)	Noisy or non-stationary problems
Adam	Combines momentum and RMSprop, with bias correction	**Default choice, learning rate 0.001**

Source: Kingma, D. & Ba, J. (2015). Adam: A Method for Stochastic Optimization. ICLR.

3. MULTILAYER PERCEPTRON

3.2. BACKPROPAGATION

Backpropagation vs Gradient Descent?

3. MULTILAYER PERCEPTRON

3.2. BACKPROPAGATION

Backpropagation vs Gradient Descent?

To put it plainly, **gradient descent** is the process of using gradients to find the minimum value of the cost function, while **backpropagation** is calculating those gradients by moving in a backward direction in the neural network. Judging from this, it would be safe to say that gradient descent relies on backpropagation.

3. MULTILAYER PERCEPTRON

3.3. ACTIVATION FUNCTIONS

- For classification, it is suitable to use the same function in all MLP neurons and restrict the outputs to the range of (0, 1) or (–1, 1).

- However, for regression it is very common to use a linear output activation function. Thus, the network may be optimized by Least Squares.

- Another frequent approach is to use the **generalized sigmoidal function** (also called ***softmax***) in the output for 1-of-*n* classification.

$$o_i^{(M)} = \phi\left(net_i^{(M)}\right) = \frac{e^{net_i^{(M)}}}{\sum_{j=1}^{J_M} e^{net_j^{(M)}}};$$

- The use of the *softmax* function introduces additional flexibility into the MLP model since the response of each output neuron is tempered by the responses of all the output neurons.

3. MULTILAYER PERCEPTRON

3.3. ACTIVATION FUNCTIONS – HIDDEN LAYERS

- Hidden layers need a **nonlinear** activation; otherwise the MLP collapses into one linear model.

Function	Formula	Range	Main issue	Typical use
Sigmoid	$\sigma(z) = 1 / (1 + e^{-z})$	(0, 1)	Saturates; gradient ≤ 0.25	Binary output layer
Tanh	$tanh(z) = 2\sigma(2z) - 1$	(−1, 1)	Saturates, but zero-centred	Shallow nets, RNNs
ReLU	$max(0, z)$	[0, ∞)	"Dying ReLU": neurons stuck at 0	**Hidden-layer by default**
Leaky ReLU	$max(\alpha z, z), \alpha \approx 0.01$	(−∞, ∞)	One more hyperparameter (α)	When many ReLUs die

ReLU's gradient is exactly 1 for z > 0, so it does not shrink as it travels backwards. This is why ReLU mitigates the **vanishing gradient problem** (3.4).

3. MULTILAYER PERCEPTRON

3.3. ACTIVATION FUNCTIONS – OUTPUT LAYER AND LOSS

- For classification, **cross-entropy** replaces MSE. It measures how far the predicted probabilities are from the true class: $E = -\Sigma_k y_k \log \hat{y}_k$
- Combined with softmax, the output gradient simplifies to $\hat{y} - y$: a confidently wrong prediction gets a large correction, so training is faster than with MSE.

Task	Output layer	Loss function	Keras loss
Regression	Linear neurons	Mean squared error	`"mse"`
Binary classification	1 sigmoid neuron	Binary cross-entropy	`"binary_crossentropy"`
Multi-class (1-of-n)	n softmax neurons	Categorical cross-entropy	`"categorical_crossentropy"`

3. MULTILAYER PERCEPTRON

3.4. OTHER CONSIDERATIONS

1. **Optimizing Network Structure**
2. Regularization
3. Vanishing Gradient Problem
4. Initializing Weights

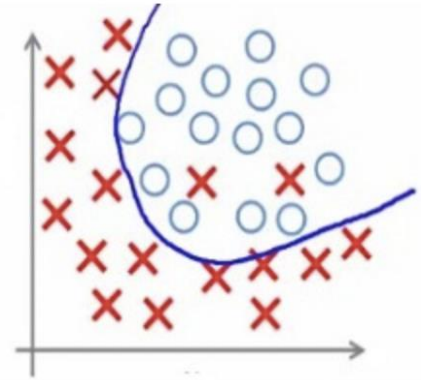
- Smaller networks that use fewer parameters usually have better generalization capability
- The number of hidden neurons and hidden layers are usually estimated by trial-and-error (Cross Validation).
- **Network pruning** strategy first selects a network with a large number of hidden units, then removes the redundant units during the learning process
- **Network growing** starts with a small network and then gradually adds hidden units until a given performance is achieved

3. MULTILAYER PERCEPTRON

3.4. OTHER CONSIDERATIONS

1. Optimizing Network Structure
2. **Regularization**
3. Vanishing Gradient Problem
4. Initializing Weights

- In machine learning, you will have a fair idea that regularization penalizes the coefficients. In deep learning, it penalizes the weight matrices of the nodes.



- There are several regularization techniques used in Deep learning:
  - L1 & L2 terms
  - Dropout
  - Data Augmentation*

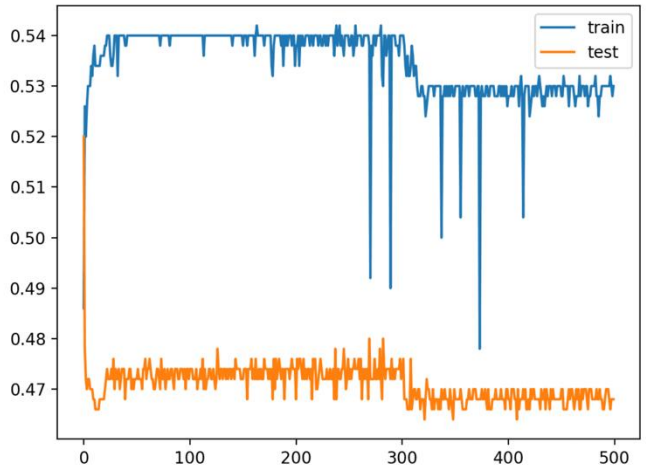
*NOTE: search more information by yourself

3. MULTILAYER PERCEPTRON

3.4. OTHER CONSIDERATIONS

1. Optimizing Network Structure
2. Regularization
3. **Vanishing Gradient Problem**
4. Initializing Weights

- The vanishing gradient problem is encountered when training artificial neural networks with gradient-based learning methods and backpropagation.
- The problem is that in some cases, **the gradient will be vanishingly small**, effectively preventing the weight from changing its value.
- In the worst case, this may completely stop the neural network from further training.



3. MULTILAYER PERCEPTRON

3.4. OTHER CONSIDERATIONS

1. Optimizing Network Structure
2. Regularization
3. Vanishing Gradient Problem
4. **Initializing Weights**

- The objective of weight initialization is to find weights that are as close as possible to a global minimum before training, and to increase the convergence speed.
- Weights must be small, zero-mean and random: identical weights make every neuron learn the same thing.
- Modern frameworks scale the variance to the layer size, so the signal neither vanishes nor explodes
- Keras uses Glorot uniform by default.
- Older studies on three-layer MLPs found a uniform range of [–0.77, 0.77] worked best.

3. MULTILAYER PERCEPTRON

3.5. TRAINING IN PRACTICE

Scale the inputs

Standardize features to mean 0 and std 1. Unscaled inputs saturate neurons and slow down gradient descent.

Split train / validation / test

Tune hyperparameters on the validation set and report the final score on the untouched test set.

Watch the learning curves

Validation loss rising while training loss keeps falling is the signature of overfitting.

Use early stopping

Stop when validation loss has not improved for k epochs (patience) and restore the best weights.

Add batch normalization

Normalizes each layer's inputs per mini-batch; allows higher learning rates and faster training.

Tune the learning rate first

The most sensitive hyperparameter: try 1e-4 to 1e-1 on a log scale before anything else.

AGENDA

1. Introduction

2. Single Layer Perceptron

3. Multilayer Perceptron
   - 3.1. Universal Approximation
   - 3.2. Backpropagation
   - 3.3. Activation Functions
   - 3.4. Other Considerations
   - 3.5. Training in Practice

4. Tensorflow & Keras

5. Summary



4. TENSORFLOW & KERAS

Tensorflow is an end-to-end open-source deep learning framework developed by Google and released in 2015. It is known for documentation and training support, scalable production and deployment options, multiple abstraction levels, and support for different platforms.



Keras is an effective high-level neural network Application Programming Interface (API) written in Python. This open-source neural network library is designed to provide fast experimentation with deep neural networks, and since Keras 3 it can run on top of TensorFlow, JAX or PyTorch (CNTK and Theano are discontinued).



Tensorflow vs Keras?

4. TENSORFLOW & KERAS

Tensorflow is an end-to-end open-source deep learning framework developed by Google and released in 2015. It is known for documentation and training support, scalable production and deployment options, multiple abstraction levels, and support for different platforms.



Keras is an effective high-level neural network Application Programming Interface (API) written in Python. This open-source neural network library is designed to provide fast experimentation with deep neural networks, and since Keras 3 it can run on top of TensorFlow, JAX or PyTorch (CNTK and Theano are discontinued).



We should bear in mind that comparing TensorFlow and Keras isn't the best way to approach the question since Keras functions as a wrapper to TensorFlow's framework. Thus, you can define a model with Keras' interface, which is easier to use, then drop down into TensorFlow when you need to use a feature that Keras doesn't have, or you're looking for specific TensorFlow functionality.

4. TENSORFLOW & KERAS

AN MLP FOR REGRESSION IN KERAS

```
import keras
from keras import layers

model = keras.Sequential([
    keras.Input(shape=(n_features,)),
    layers.Dense(64, activation="relu",
                 kernel_initializer="he_normal"),
    layers.Dropout(0.2),
    layers.Dense(1),            # linear output
])
model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss="mse", metrics=["mae"])
stop = keras.callbacks.EarlyStopping(
    patience=10, restore_best_weights=True)
history = model.fit(X_train, y_train, epochs=200,
    batch_size=32, validation_split=0.2, callbacks=[stop])
model.evaluate(X_test, y_test)
```

1. **Hidden layers:** Dense + ReLU, with He initialization
2. **Dropout** switches off 20% of the neurons at each step to regularize
3. **Output:** one linear neuron, because the target is a price
4. **Compile:** Adam optimizer, MSE loss, MAE as the readable metric
5. **Fit:** mini-batches of 32 and early stopping on a 20% validation split

AGENDA

1. Introduction

2. Single Layer Perceptron

3. Multilayer Perceptron
   - 3.1. Universal Approximation
   - 3.2. Backpropagation
   - 3.3. Activation Functions
   - 3.4. Other Considerations
   - 3.5. Training in Practice

4. Tensorflow & Keras

5. Summary



5. SUMMARY

WHEN TO USE AN MLP

Data type	Best architecture	Why
Tabular data	**MLP, or gradient boosting**	Features have no spatial or time order; tree ensembles are the baseline to beat
Images	Convolutional networks (CNN)	An MLP ignores spatial structure: a 224×224 RGB image needs 150,528 weights per first-layer neuron
Sequences and text	RNNs and Transformers	An MLP needs a fixed-size input and has no notion of order

- MLPs also need more data and tuning than classical models, and their decisions are **hard to interpret**.

5. SUMMARY

KEY TAKEAWAYS

1. One neuron is a linear classifier. **Hidden layers with nonlinear activations** make the MLP a universal approximator.

2. **Backpropagation** computes the gradients with the chain rule; gradient descent uses them to update the weights.

3. Match output layer and loss to the task: **linear + MSE, sigmoid + binary CE, softmax + categorical CE**.

4. Modern defaults: **ReLU, He initialization, Adam and mini-batches**.

5. Always **scale the inputs** and control overfitting with regularization, dropout and early stopping.

PROBLEM

PROBLEM

1) In **session 7** we built models to predict house prices. This is a **regression** problem. Train a Multilayer Perceptron and compare the results with the previous models.
2) Scale the input features (e.g. StandardScaler) and use a single linear output neuron with MSE loss.
3) Try at least two architectures (number of hidden layers and neurons) and use early stopping on a validation split.
4) Compare all models on the same test set using **RMSE** and **MAE**, and plot the training and validation loss curves.

REFERENCES

- McCulloch, W. & Pitts, W. (1943). *A logical calculus of the ideas immanent in nervous activity*. Bulletin of Mathematical Biophysics.
- Rosenblatt, F. (1958). *The perceptron: a probabilistic model for information storage and organization in the brain*. Psychological Review.
- Minsky, M. & Papert, S. (1969). *Perceptrons*. MIT Press.
- Rumelhart, D., Hinton, G. & Williams, R. (1986). *Learning representations by back-propagating errors*. Nature.
- Cybenko, G. (1989). *Approximation by superpositions of a sigmoidal function*. Mathematics of Control, Signals and Systems.
- Glorot, X. & Bengio, Y. (2010). *Understanding the difficulty of training deep feedforward neural networks*. AISTATS.
- He, K., Zhang, X., Ren, S. & Sun, J. (2015). *Delving deep into rectifiers*. ICCV.
- Kingma, D. & Ba, J. (2015). *Adam: a method for stochastic optimization*. ICLR.
- Goodfellow, I., Bengio, Y. & Courville, A. (2016). *Deep Learning*. MIT Press.
- Keras documentation: *keras.io*


THANK YOU